



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 6
по курсу «Языки и методы программирования»
«Графические приложения на Java Swing»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Приложение PictureForm	1
Структура приложения	1
Класс CanvasPanel	1
Главный класс PictureForm	2
Приложение RubiksCubeApp	4
Класс Sticker	4
Класс CubeCanvas	6
Главное окно приложения	7
Сборка и запуск	9
PictureForm	9
RubiksCubeApp	9
Вывод	10

Цель работы

Разработать графические приложения на Java с использованием библиотеки Swing.

Создать приложение для рисования геометрических фигур (окружностей и квадратов) и 3D-модель кубика Рубика с возможностью вращения.

Задачи:

- Изучить основы библиотеки Swing.
- Создать приложение PictureForm с настраиваемыми параметрами рисования.
- Разработать приложение RubiksCubeApp с 3D-визуализацией кубика.
- Реализовать вращение 3D-модели вокруг осей X, Y, Z.

Приложение PictureForm

Приложение PictureForm представляет собой графическую программу для отображения окружности с расположенными вокруг неё квадратами. Параметры фигур можно изменять в реальном времени с помощью элементов управления.

Структура приложения

Приложение состоит из двух классов:

- PictureForm — главный класс с управляющими элементами (Spinner, JColorChooser)
- CanvasPanel — панель для рисования

Класс CanvasPanel

Класс CanvasPanel наследуется от JPanel и переопределяет метод paintComponent() для отрисовки графики.

CanvasPanel.java

```
class CanvasPanel extends JPanel {
    private int radius = 20;
    private int side = 20;
    private Color squareColor = Color.RED;
    private Color circleColor = Color.GREEN;

    protected void paintComponent(Graphics g) {
        super.paintComponent(g);

        Graphics2D g2d = (Graphics2D) g;

        int centerX = getWidth() / 2;
        int centerY = getHeight() / 2;

        int numSquares = (int) Math.floor(2.0 * Math.PI * radius /
side);

        g2d.setColor(circleColor);
        g2d.fillOval(centerX - radius, centerY - radius, radius *
2, radius * 2);

        for (int i = 0; i < numSquares; i++) {
            double polarAngle = 2 * Math.PI * i / numSquares;

            double x = centerX + radius * Math.cos(polarAngle);
            double y = centerY + radius * Math.sin(polarAngle);

            AffineTransform oldTransform = g2d.getTransform();

            g2d.translate(x, y);
            g2d.rotate(polarAngle);

            g2d.setColor(squareColor);
            g2d.fillRect(0, -side / 2, side, side);

            g2d.setTransform(oldTransform);
        }
    }
}
```

Главный класс PictureForm

Главный класс создаёт интерфейс пользователя с элементами управления:

PictureForm.java

```
public class PictureForm {
    private JSpinner radiusSpinner;
    private JSpinner sideSpinner;
    private CanvasPanel canvasPanel;

    public PictureForm() {
        radiusSpinner.addChangeListener(new ChangeListener() {
            @Override
            public void stateChanged(ChangeEvent changeEvent) {
                int radius = (int) radiusSpinner.getValue();
                canvasPanel.setRadius(radius);
                canvasPanel.repaint();
            }
        });

        sideSpinner.addChangeListener(new ChangeListener() {
            @Override
            public void stateChanged(ChangeEvent changeEvent) {
                int side = (int) sideSpinner.getValue();
                canvasPanel.setSide(side);
                canvasPanel.repaint();
            }
        });

        radiusSpinner.setValue(150);
        sideSpinner.setValue(30);
    }

    public void createUIComponents() {
        JColorChooser squareColorChooser = new
        JColorChooser(squareColor);
        squareColorJPanel.add(squareColorChooser);

        squareColorChooser.getSelectionModel().addChangeListener(...);
    }

    public static void main(String[] args) {
        JFrame frame = new JFrame("PictureForm");
        frame.setContentPane(new PictureForm().mainPanel);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.pack();
        frame.setVisible(true);
    }
}
```

Элементы управления:

- radiusSpinner — регулировка радиуса окружности
- sideSpinner — регулировка размера квадратов

- `JColorChooser` — выбор цвета окружности и квадратов

Приложение `RubiksCubeApp`

Приложение `RubiksCubeApp` реализует 3D-визуализацию кубика Рубика с использованием библиотеки `FlatLaF` для современного внешнего вида.

Класс `Sticker`

Класс `Sticker` представляет одну грань кубика и содержит:

- цвет грани
- координаты центра
- нормаль грани
- вершины четырёхугольника

Sticker.java

```
public class Sticker {
    public Color color;
    public double[] center;
    public double[] normal;
    public double[][] vertices;

    public void rotateX(double angleDegrees) {
        double rad = Math.toRadians(angleDegrees);
        double cos = Math.cos(rad);
        double sin = Math.sin(rad);
        rotatePointX(this.center, cos, sin);
        rotatePointX(this.normal, cos, sin);
        for (double[] v : vertices)
            rotatePointX(v, cos, sin);
    }

    public void rotateY(double angleDegrees) {
        double rad = Math.toRadians(angleDegrees);
        double cos = Math.cos(rad);
        double sin = Math.sin(rad);
        rotatePointY(this.center, cos, sin);
        rotatePointY(this.normal, cos, sin);
        for (double[] v : vertices)
            rotatePointY(v, cos, sin);
    }

    public void rotateZ(double angleDegrees) {
        double rad = Math.toRadians(angleDegrees);
        double cos = Math.cos(rad);
        double sin = Math.sin(rad);
        rotatePointZ(this.center, cos, sin);
        rotatePointZ(this.normal, cos, sin);
        for (double[] v : vertices)
            rotatePointZ(v, cos, sin);
    }

    public int[][] getScreenCoordinates( ... ) {
        if (this.normal[2] < 0.001)
            return null;
        // проецирование 3D → 2D
    }
}
```

Методы вращения используют матрицы поворота:

- rotateX() — вращение вокруг оси X
- rotateY() — вращение вокруг оси Y
- rotateZ() — вращение вокруг оси Z

Метод `getScreenCoordinates()` выполняет перспективное проецирование 3D-координат на 2D-экран.

Класс `CubeCanvas`

Класс `CubeCanvas` управляет генерацией и отрисовкой кубика:

RubiksCubeApp.java

```
private static class CubeCanvas extends JPanel {
    private int cubeSize = 3;
    private List<Sticker> baseCubeStickers;

    private static final Color[] COLORS = {
        new Color(0, 81, 186),    // синий
        new Color(0, 158, 96),    // зелёный
        new Color(255, 255, 255), // белый
        new Color(255, 213, 0),   // жёлтый
        new Color(196, 30, 58),   // красный
        new Color(255, 88, 0)    // оранжевый
    };

    private void generateCube(int size) {
        baseCubeStickers = new ArrayList<>();
        double offset = size / 2.0;
        double r = 0.46;

        for (int face = 0; face < 6; face++) {
            for (int i = 0; i < size; i++) {
                for (int j = 0; j < size; j++) {
                    // генерация граней кубика
                }
            }
        }
    }

    @Override
    protected void paintComponent(Graphics g) {
        // сортировка по глубине и отрисовка
        frameStickers.sort(Comparator.comparingDouble(s →
        s.center[2]));

        for (Sticker s : frameStickers) {
            int[][] coords = s.getScreenCoordinates(...);
            if (coords != null) {
                g2d.setColor(s.color);
                g2d.fillPolygon(xPoints, yPoints, 4);
            }
        }
    }
}
```

Главное окно приложения

RubiksCubeApp.java

```

public class RubiksCubeApp extends JFrame {
    private JSpinner sizeSpinner;
    private JSpinner angleXSpinner;
    private JSpinner angleYSpinner;
    private JSpinner angleZSpinner;
    private JSpinner cameraDistanceSpinner;
    private JSpinner scaleSpinner;

    public RubiksCubeApp() {
        setTitle("Кубик Рубика");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(800, 600);

        canvas = new CubeCanvas();
        JPanel controlPanel = createControlPanel();
        add(controlPanel, BorderLayout.NORTH);
        add(canvas, BorderLayout.CENTER);
    }

    private JPanel createControlPanel() {
        // создание панелей управления
        sizeSpinner = new JSpinner(new SpinnerNumberModel(3, 1,
100, 1));
        angleXSpinner = new JSpinner(new SpinnerNumberModel(120, 0,
3600, 1));
        angleYSpinner = new JSpinner(new SpinnerNumberModel(145, 0,
3600, 1));
        angleZSpinner = new JSpinner(new SpinnerNumberModel(90, 0,
3600, 1));

        JButton shuffleButton = new JButton("Перемешать цвета");
        shuffleButton.addActionListener(e → {
            canvas.shuffleColors();
            canvas.repaint();
        });
    }

    public static void main(String[] args) {
        try {
            UIManager.setLookAndFeel(new FlatDarkLaf());
        } catch (Exception e) {
            e.printStackTrace();
        }
        SwingUtilities.invokeLater(() → {
            RubiksCubeApp app = new RubiksCubeApp();
            app.setVisible(true);
        });
    }
}

```

Элементы управления:

- sizeSpinner — размер кубика (1-100)
- angleXSpinner, angleYSpinner, angleZSpinner — углы поворота
- cameraDistanceSpinner — расстояние до камеры
- scaleSpinner — масштаб отображения
- shuffleButton — перемешивание цветов

Сборка и запуск

PictureForm

Компиляция и запуск:

terminal

```
$ javac PictureForm.java
$ java PictureForm
```

RubiksCubeApp

Для сборки используется Makefile:

Makefile

```
.PHONY: all clean run

all:
    javac -cp lib/flatlaf-3.7.jar src/*.java

run:
    java -cp .:lib/flatlaf-3.7.jar RubiksCubeApp

clean:
    rm -f src/*.class
```

terminal

```
$ cd dop
$ make
$ make run
```

Результат — окно с 3D-кубиком Рубика, который можно вращать вокруг осей X, Y, Z с помощью регуляторов.

Вывод

В ходе лабораторной работы были разработаны два графических приложения на Java Swing:

1. PictureForm — приложение для рисования окружности с квадратами, расположенными по периметру. Реализованы элементы управления для изменения радиуса, размера квадратов и выбора цветов.
2. RubiksCubeApp — приложение для 3D-визуализации кубика Рубика. Реализованы:
 - генерация кубика заданного размера
 - вращение вокруг осей X, Y, Z
 - перспективное проецирование
 - сортировка граней по глубине (z-buffer)
 - перемешивание цветов

Оба приложения демонстрируют основные принципы работы со Swing: использование JFrame, JPanel, JSpinner, JColorChooser, JButton и обработка событий.